A few standards have emerged for implementing distributed tracing and defining guidelines for generating and propagating traces and spans. OpenZipkin is the more mature project (https://zipkin.io). OpenTracing and OpenCensus are more recent projects that have tried to standardize ways of instrumenting application code to support distributed tracing. They are both deprecated now, since they joined forces to work on OpenTelemetry: the ultimate framework to "instrument, generate, collect, and export telemetry data (metrics, logs, and traces)." Tempo supports all those options.

Spring Cloud Sleuth (https://spring.io/projects/spring-cloud-sleuth) is a project that provides auto-configuration for distributed tracing in Spring Boot applications. It takes care of instrumenting commonly used libraries in Spring applications and provides an abstraction layer on top of specific distributed tracing libraries. OpenZipkin is the default choice.

In this book, I decided to show you how to use the OpenTelemetry Java instrumentation directly for two main reasons. First, support for OpenTelemetry in Spring Cloud Sleuth is still experimental and not ready for production at the time of writing (https://github.com/spring-projects-experimental/spring-cloud-sleuth-otel).

Second, Spring Cloud Sleuth will not be developed further once Spring Framework 6 and Spring Boot 3 are released. The Spring project donated the Sleuth core framework to Micrometer and created a new Micrometer Tracing subproject aiming to provide a vendor-neutral façade for traces, similar to what Micrometer already does for metrics. Micrometer Tracing will provide support for OpenZipkin and OpenTelemetry. Based on Micrometer Tracing, code instrumentation will become a core aspect of all Spring libraries as part of the Spring Observability initiative.

13.4.1 Managing traces with Tempo and Grafana

A distributed tracing backend is responsible for aggregating, storing, and making traces searchable. Tempo is the solution in the Grafana observability stack. Figure 13.8 illustrates the tracing architecture.